

Odette Java GUI Documentation

Satellite Team

June 5, 2025

Contents

1	Java Graphical User Interface	1
1.1	Installation Instructions	1
1.2	Specification and Usage Instructions	3
1.3	Implementation	6
1.3.1	GuiMain	7
1.3.2	BackgroundGrid	7
1.3.3	LeftPanel	7
1.3.4	ParameterPanel	8
1.3.5	MultipleChoicePanel	8
1.3.6	TwoChoicePanel	8
1.3.7	OnOffButtons	8
1.3.8	RightPanel	9
1.3.9	PathRegistry	9
1.3.10	Choice	9

1 Java Graphical User Interface

1.1 Installation Instructions

Note: In their current form, the Java GUI files ensure Windows functionality. Users with other operating systems might encounter some compatibility issues, especially coming from the Python-C++ binding.

The Odette graphical interface can be installed as a standalone application by downloading the `Java-Interface.zip` archive from GitHub: https://github.com/teomatei22/odette_c/tree/Java-Interface. It is not necessary to download the C++ and Python files separately, as they are already included in the archive.

After unzipping the archive, the user obtains two folders: `odette_c-python-interface`, which contains the C++ and Python files, and `JavaInterface`, which contains the GUI files. The GUI source files are found under `JavaInterface\JavaProject\src\graphical_interface`, with `GuiMain.java` being the driver class of this application. Inside the `JavaInterface` folder we also notice the `Data` folder, which contains example input files for orbit simulation, as well as the `Plots` folder, which stores the plots generated in the simulation.

In order to get the application running, there are 3 `.jar` files that need to be included in the Java project created by the user (Important: the Java project should be created in the `JavaProject` folder, using the files provided). These files are found in the `JavaInterface` folder: `flatlaf-3.6.jar`, `jep-4.2.2.jar`, `json-20250517.jar`. They allow us to import libraries responsible for changing the Look and Feel of the interface for a more modern design, to create the Java-Python binding, and respectively to work with JSON objects. For those using Eclipse as their IDE, the process is quite simple. For other IDEs, specific instructions might be found online.

In Eclipse, we begin by creating a new project based on the provided source files. Next, we right-click on the project name in the left-side bar, and then go to Build Path → Configure Build Path → Java Build Path → Libraries. We click on Classpath, and then on Add External JARs, to add the JAR files individually. For the JEP JAR file, there is one additional step: we expand the JEP line, and go to Native Library Location, and then click edit. Here, we select the folder `jep` from the `JavaInterface` parent folder contained in the downloaded archive. With these settings in place, the graphical interface can be started by running the `GuiMain.java` file from the IDE.

Name	Date	Type	Size
Data	23-May-25 17:49	File folder	
JavaProject	23-May-25 14:44	File folder	
jep	23-May-25 15:51	File folder	
Plots	23-May-25 16:22	File folder	
flatlaf-3.6.jar	15-Apr-25 18:52	Executable Jar File	904 KB
jep-4.2.2.jar	10-May-25 9:31	Executable Jar File	165 KB
json-20250517.jar	24-May-25 12:45	Executable Jar File	81 KB

Figure 1: Structure of the JavaInterface folder. In the user's IDE, the project files should be placed inside the JavaProject folder.

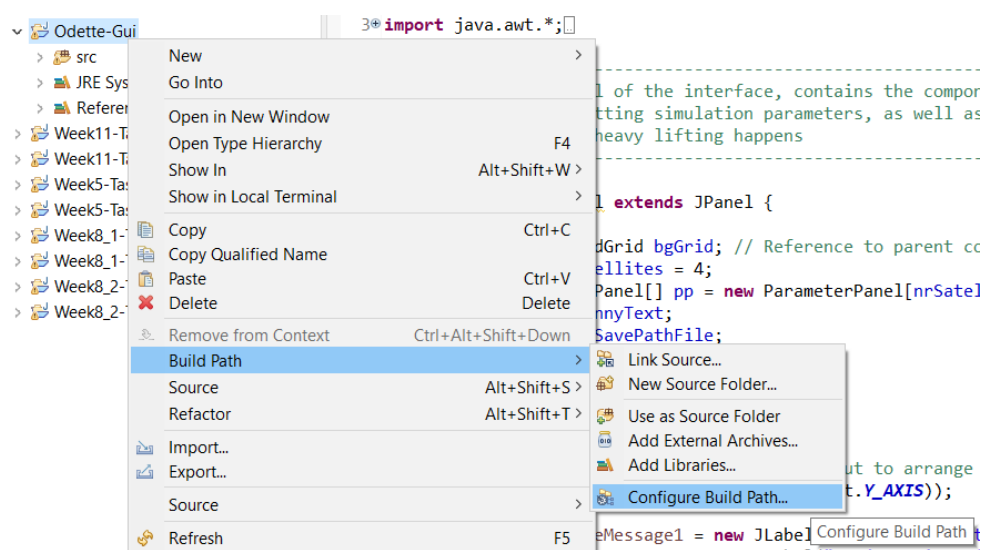


Figure 2: Configuring the project build path in Eclipse.

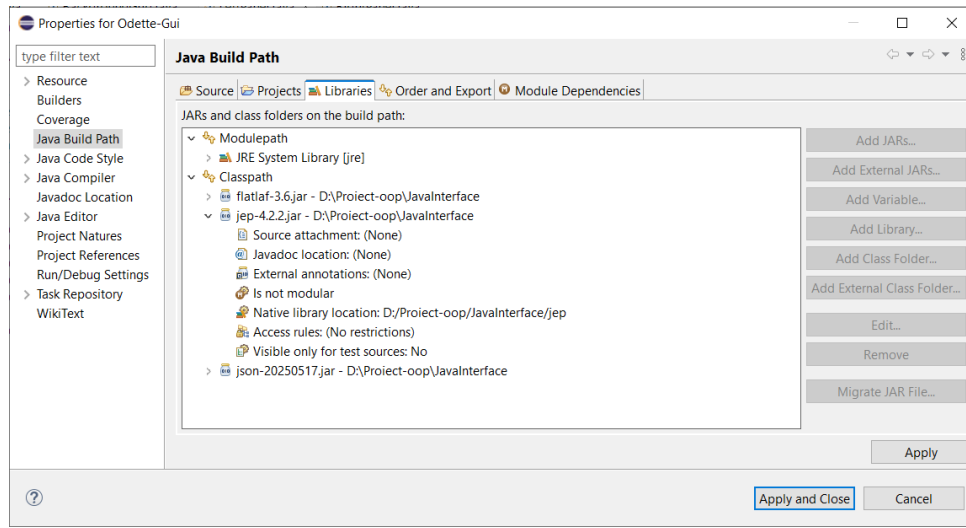


Figure 3: Adding external JAR files in Eclipse.

1.2 Specification and Usage Instructions

The Odette graphical interface allows the user to access the computational capacity of our C++ and Python software without requiring any programming knowledge. With a clean and simple layout, the interface allows the simulation of up to 4 satellites at the same time, requiring minimal effort.

The interface is made up of three main areas: upon launching, the user is met with two windows: the left window is meant for setting up simulation parameters, such as input files, desired plot types, and orbit perturbations; the right window displays plots of orbital elements corresponding to each individual simulated satellite. After running the simulation, the right window is responsible with displaying the generated 2 dimensional plots, which can be selected from the two ComboBoxes present on the right panel. Again, after running the simulation, the third window is opened next to the right window. This will contain a 3 dimensional representation of the satellite orbit, which can be rotated with the mouse in order to inspect the trajectory from various angles.

The steps to perform an orbit simulation are as follows:

- Select satellite to simulate from the list Sat1 - Sat4.
- For a given satellite, select desired input files in TDM or TLE format using the Browse Files button.
Please Note: when simulating orbits with TDM input files, the program has to be provided with at least 3 TDM files associated to the satellite's motion, and these files have to be placed inside a folder - the folder is then chosen as input file for the satellite. For TLE input, the data has to be placed inside a JSON file. Consult the example files in the Data folder for more details.
- Select type of input file for a given satellite.
- Select desired plot types for a given satellite: 2D or 3D plotting.
- Select desired perturbations for a given satellite. The list of possible perturbations is comprised of: air drag, solar radiation pressure, third body perturbation, and J2 perturbation.
- After choosing simulation parameters for up to 4 satellites, press the Take Off button.

After a few seconds, the user should notice the plots that appear in the right window (now the central window, after the appearance of the 3D plot). Next to it, we have the 3 dimensional plot of the satellite orbit.

Due to the nature of the underlying computational code, we can only generate a 3D representation for up to 1 TDM file and 1 TLE file at the same time. There are no restrictions for the 2D plots, however. If the user requests the generation of a 3D plot for more than 1 TDM and 1 TLE input files at the same time, they will be met with a pop-up message on the screen, explaining the maximum accepted number of input files.

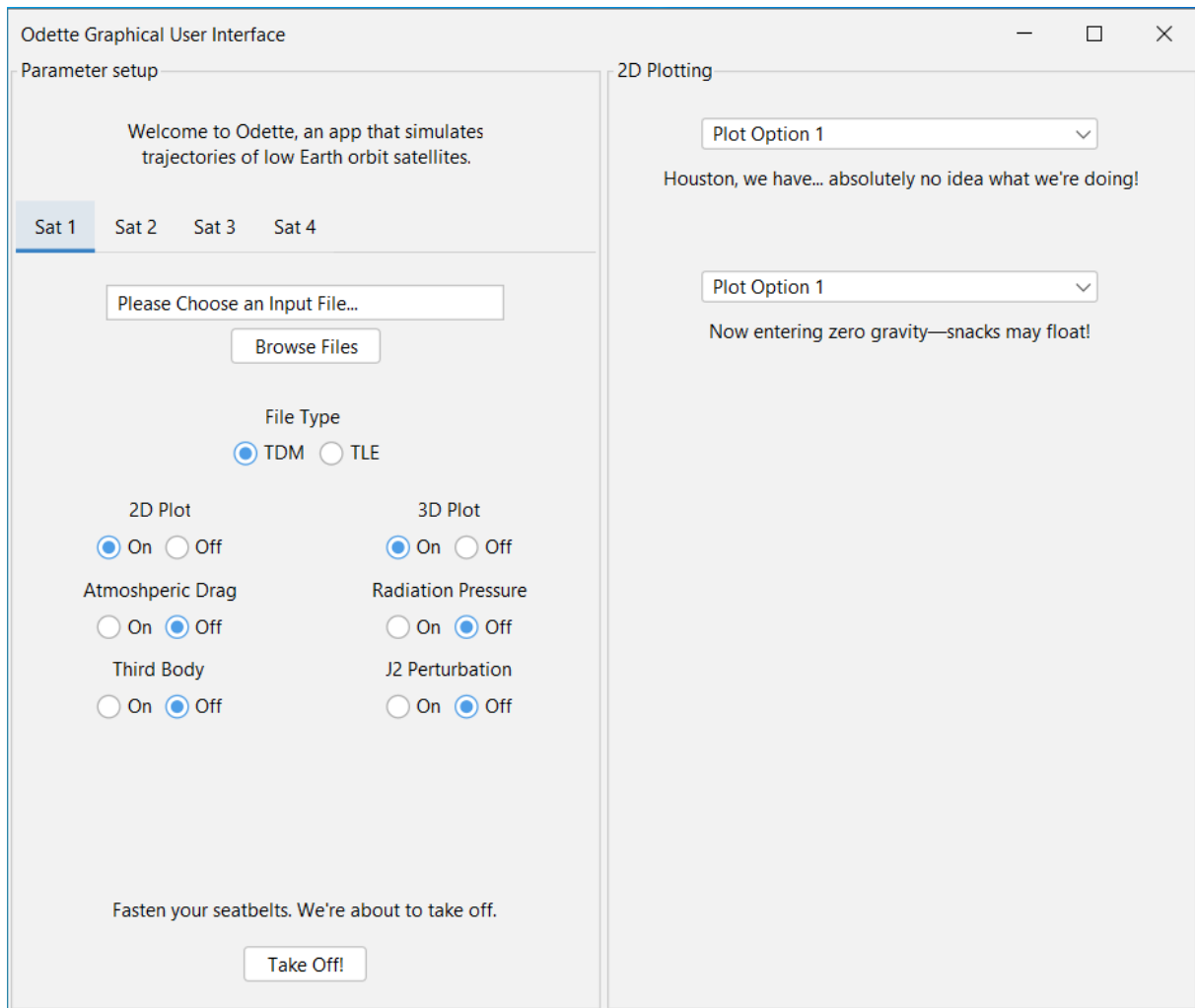


Figure 4: The Odette graphical interface upon opening.

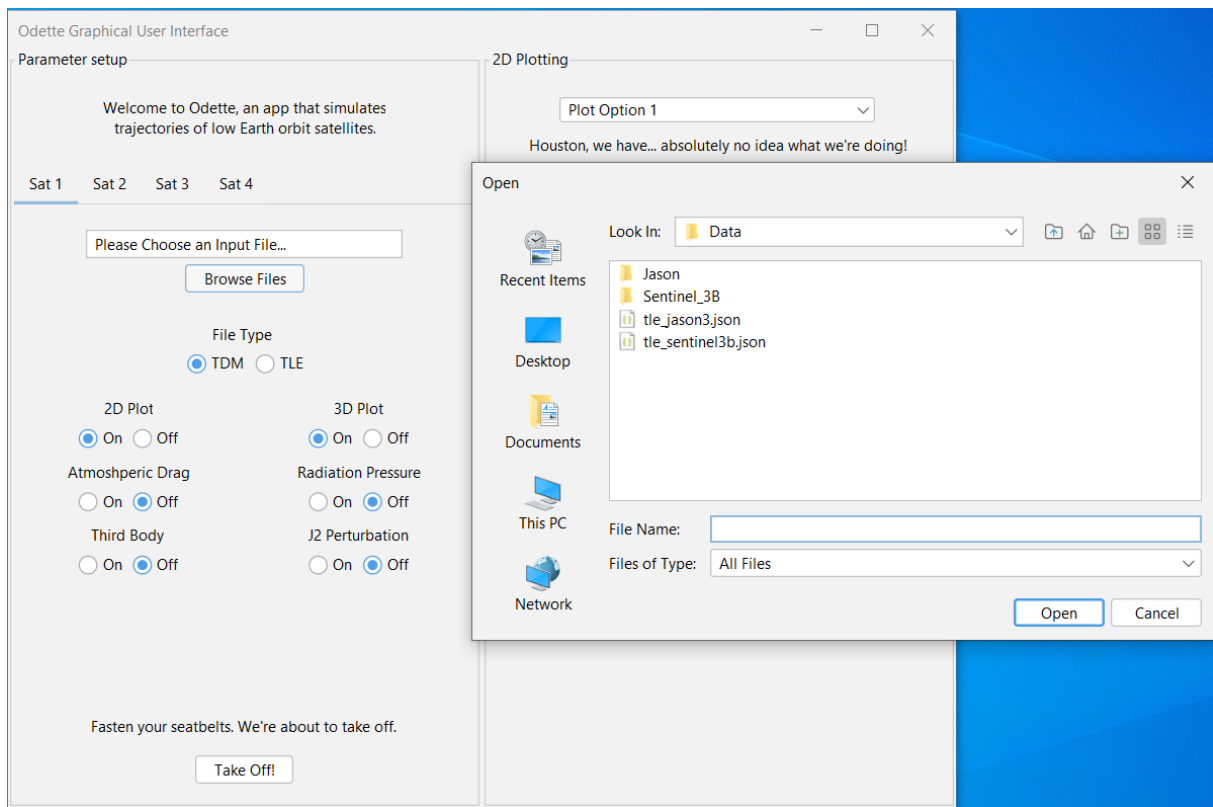


Figure 5: Choosing the simulation input files.

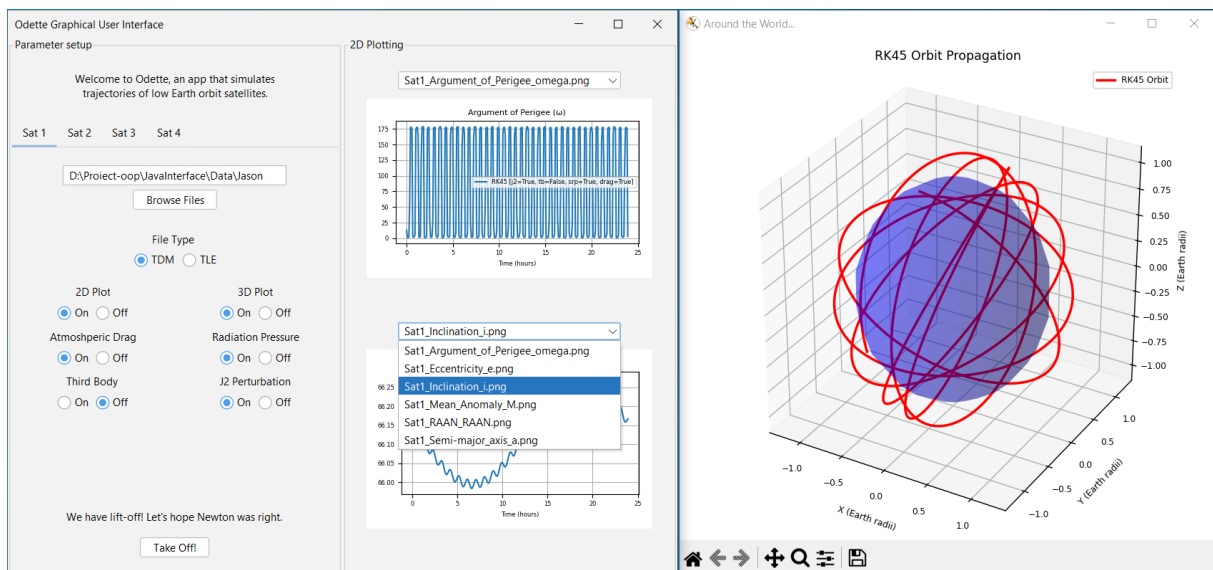


Figure 6: The graphical interface upon successfully running an orbit simulation.

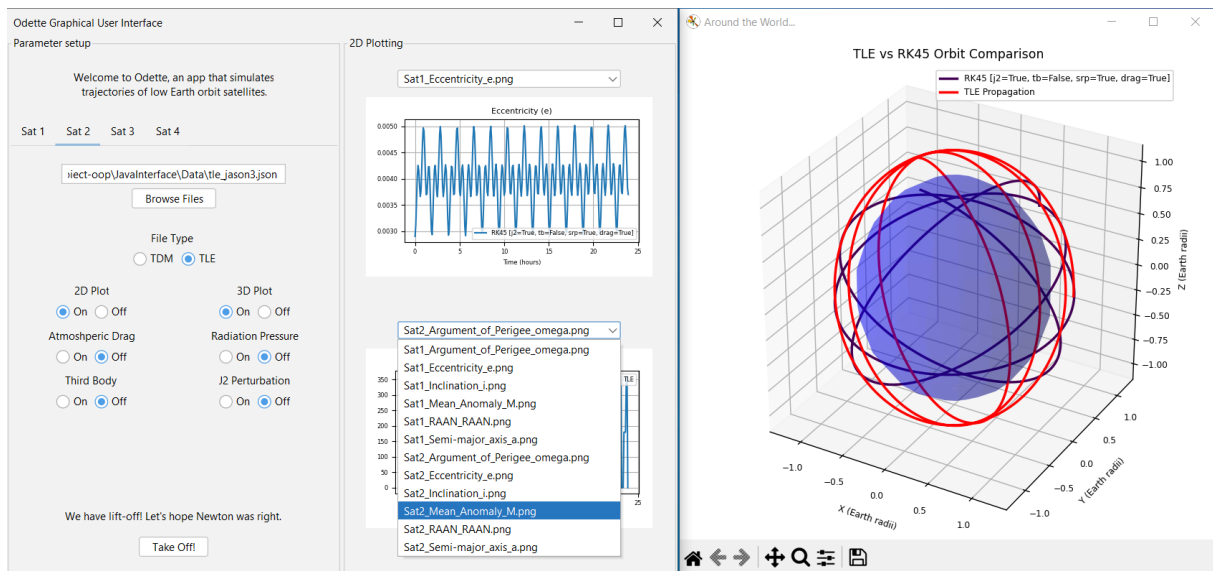


Figure 7: Simulating TDM and TLE orbits at the same time.

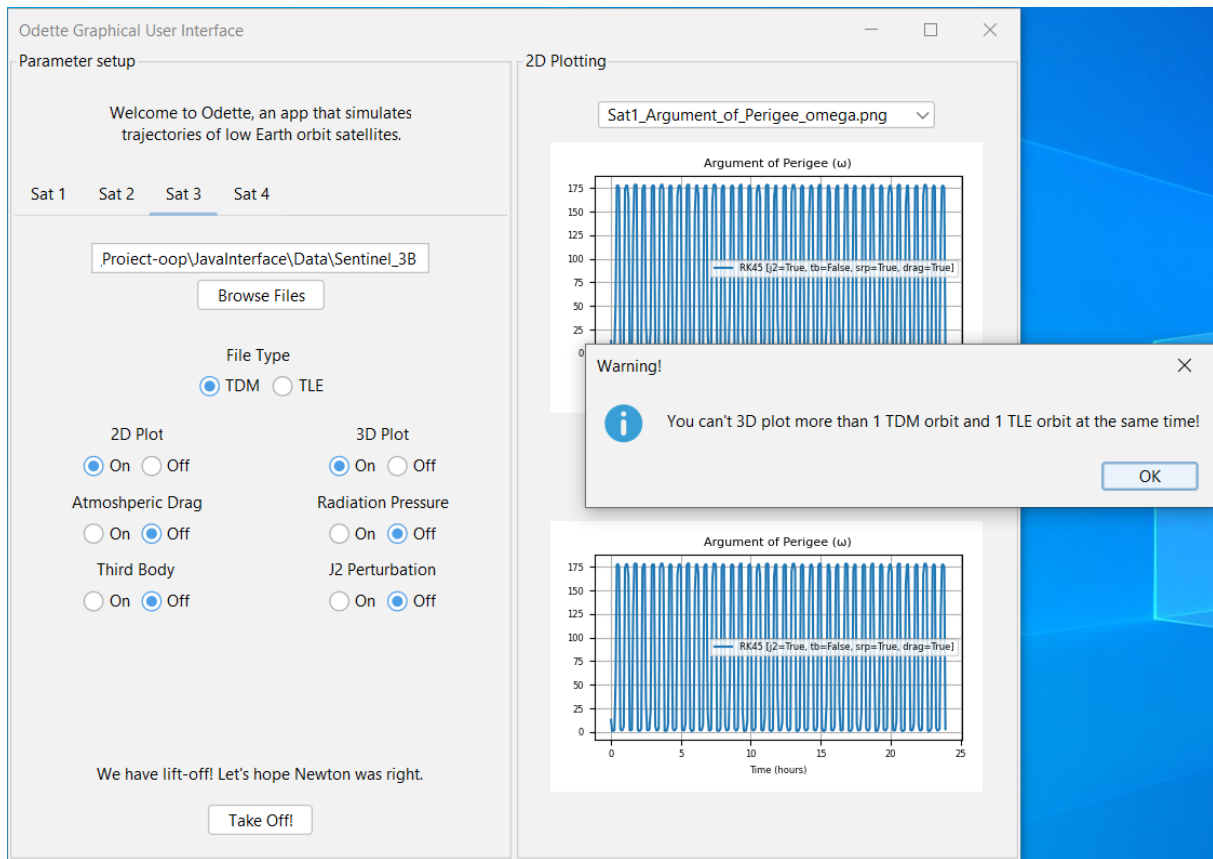


Figure 8: Error encountered when maximum number of satellites for 3D plotting is crossed.

1.3 Implementation

The Java code for the Odette graphical interface consists in 9 classes, together with one interface that is implemented by some of the classes. The classes vary in the complexity of the code they contain, as

well as the tasks that they perform. Below, we provide a list of the Java classes, with brief explanations regarding their functionality.

1.3.1 GuiMain

This is the driver class for the Odette graphical interface. For the most part, `GuiMain` contains standard code used for starting up a graphical interface. One important feature is that here we change the Look and Feel of the interface for a more modern aspect of our application. The LaF we have used is the `FlatLightLaf`, imported from the `flatlaf` package, as detailed in the section regarding dependencies.

1.3.2 BackgroundGrid

The `BackgroundGrid` class is again straightforward, representing a `Panel` that uses the grid layout in order to have good control over the positions of the main areas of our interface: the left panel and the right panel, which are instantiated inside the `BackgroundGrid`. One important aspect is that this class enables the communication between the left and right panels, such that when the start button is pressed on the left panel, the resulting plots are displayed on the right panel. This is achieved by passing a reference to the `BackgroundGrid` class in the `LeftPanel` class. In this way, the left panel can call the `updateComboBoxes()` method of the `BackgroundGrid`, which then sends the command further to the right panel.

Methods:

- `public void updateComboBoxes()` - This method is called by the left panel so that the combo boxes on the right panel are updated when new plots are generated by pressing the Take Off button on the left panel. The `BackgroundGrid` class mediates this process.

1.3.3 LeftPanel

The `LeftPanel` is the most important part of the graphical interface. It contains a `ParameterPanel` for each satellite, through which we individually set the parameters for the orbit simulation. The `LeftPanel` contains the Take Off button, which starts the simulation when it is pressed, by calling the `runForrest()` method, detailed below.

Methods:

- `private ArrayList<HashMap<String, Boolean>> buildPerturbation(ParameterPanel pp)` - This method is responsible for collecting the perturbation parameters set on each `ParameterPanel` and putting them together in an `ArrayList` which contains one `HashMap` element, with keys corresponding to the perturbation names, and values given by the true/false status of the perturbations.
- `private String[] getTleLines(String filePath)` - Method used for reading a JSON file with TLE data and returning the TLE information as a list of two strings.
- `private JSONObject buildPlotCommand(int dimension, String plotType, int i)` - Method responsible for creating the `JSONObject` that is sent as input to the Python functions for plotting. It takes as input the dimension of the required plot (2D/3D), the type of the plot, and the index `i` corresponding to the `ParameterPanel` from which we take the simulation parameters.

- `private void runForrest()` - Method run when the Take Off button is pressed. It goes through all the `ParameterPanels` to collect the simulation parameters and proceeds to call the Python functions responsible for plotting. Here, the connection between Java and Python is realized through the JEP library (Java Embedded Python). More specifically, we use the `SharedInterpreter` class from JEP to call the Python plotting functions.

1.3.4 ParameterPanel

The `ParameterPanel` contains components used to set simulation parameters for each satellite: choice buttons and a text field for the file path, together with a file browsing button. The `ParameterPanel` is based on the `MultipleChoicePanel` class.

Methods:

- `public boolean[] getChoices()` - Returns a list of the boolean values corresponding to the chosen simulation parameters: types of chosen plots and the four orbit perturbations.
- `public boolean[] getFileType()` - Returns a list with a single boolean value corresponding to the chosen input file type. True corresponds to TDM files, while False corresponds to TLE files.
- `public String getFilePath()` - Returns a String representing the absolute path to the chosen input file.

1.3.5 MultipleChoicePanel

The `MultipleChoicePanel` groups together `TwoChoicePanel` objects used for setting each of the individual simulation parameters.

Methods:

- `public boolean[] getChoices()` - Returns a list of the boolean values corresponding to the chosen simulation parameters: types of chosen plots and the four orbit perturbations.

1.3.6 TwoChoicePanel

The `TwoChoicePanel` is used to set each simulation parameter; it contains the name of the parameter and a set of on/off buttons.

Methods:

- `public boolean[] getChoices()` - Returns a list of a single boolean value corresponding to the on/off state of the chosen simulation parameter.

1.3.7 OnOffButtons

Create a pair of on/off buttons to set each simulation parameter.

Methods:

- `public boolean[] getChoices()` - Returns a list of a single boolean value corresponding to the on/off state of the chosen simulation parameter.

1.3.8 RightPanel

This class represents the right window of the interface, responsible for displaying the generated plots for the orbital elements of the satellite orbits. The Right Panel contains two combo boxes that list all the generated plots from which the user can choose. In this way, orbital elements of different satellite orbits can be compared, for example.

Methods:

- `public void updateComboBoxOptions()` - Method responsible for updating the combo boxes with the newly generated plots each time a new simulation is run. The plots are saved in a folder, and then opened into the interface. This method is run from the LeftPanel when pressing the Take Off button, using the BackgroundGrid as an intermediary, since both LeftPanel and RightPanel are instantiated inside the BackgroundGrid.
- `private void showSelectedPlot()` - Method that displays selected plot when an option is chosen in one of the combo boxes.

1.3.9 PathRegistry

Singleton class used for storing various system paths used by the other classes for file processing.

Methods:

- `public static PathRegistry getInstance()` - Returns the single instance of the Singleton class.
- `public String getProjectDir()` - Returns a String representing the absolute path of the Java project directory.
- `public String getPlotDir()` - Returns a String representing the absolute path of the directory where generated plots are saved to be later displayed on the right panel.
- `public String getPythonDir()` - Returns a String representing the absolute path of the directory of the Python module whose functions we call for computing orbits.
- `public String getDataDir()` - Returns a String representing the absolute path of the directory where simulation input files are stored.

1.3.10 Choice

Interface defined for implementing the **Composite Design Pattern**.

Methods:

- `public boolean[] getChoices()` - Abstract method implemented by a number of classes: `OnOffButtons`, `TwoChoicePanel`, `MultipleChoicePanel`, `ParameterPanel`. In this way, we achieve the structure of leaf and container elements characteristic to the Composite pattern.